

Memory vs. State

A fixed-size recurrent state can process a sequence of any length without growing its memory — but it pays by losing the ability to perfectly retrieve any past token on demand.

THE PROBLEM

Every standard Transformer stores a key vector and value vector for every token it has processed. That key-value (KV) cache grows linearly: a sequence of n tokens requires $O(n \cdot d)$ memory and $O(n)$ time per new token to attend over. At long contexts this becomes the dominant memory cost and a hard engineering constraint for deployment.

THE MECHANISM: LINEAR ATTENTION

Linear attention replaces the softmax kernel $\exp(q \cdot k)$ with a feature map $\phi(q) \cdot \phi(k)$, decoupling the query-key product so the sum over tokens can be computed incrementally. The resulting recurrence:

$$S \leftarrow S + k \cdot v$$
$$o = (S \cdot q) / (z \cdot q), \text{ where } z \leftarrow z + k$$

collapses the entire context into a single $d \times d$ state matrix S and a d -dimensional normalizer z . Memory cost is $O(d^2)$ — constant, regardless of sequence length. Each new token is a Hebbian write (outer-product addition); each read is a linear associative retrieval.

ARCHITECTURAL COMPARISON

	Softmax Attention	Linear Attention	BDH / BDH-CQ
Memory per token	$O(n \cdot d)$ — grows	$O(d^2)$ — fixed	$O(d^2)$ — fixed
Retrieval quality	Exact	Approximate	Approximate + sparse
Inference cost	$O(n)$ per step	$O(1)$ per step	$O(1)$ per step
Long-context	Expensive	Free	Free
Test-time adapt	No	No	Yes (BDH-CQ, recurrent)
Kernel	$\exp(q \cdot k / \sqrt{d})$	$q \cdot k$ (linear)	$q \cdot k$ + sparse ReLU-low-rank

ROLE OF BDH AND BDH-CQ

Dragon Hatchling (BDH) is Pathway's Post-Transformer architecture that treats the recurrent state as the primary computational substrate, not an optimization. Key distinctions from vanilla linear attention:

- **Sparse non-negative activations.** Roughly 5% of neurons fire per token, reducing write density and mitigating interference in the shared state matrix.
- **Monosemantic synapses.** Connections reportedly encode single semantic concepts rather than distributed representations, improving state interpretability.
- **BDH-CQ** removes evaluation-task demonstrations from training and performs adaptation entirely through the recurrent state at inference — no backward pass, no parameter update. The state is the adapter.
- **Scale evidence.** Publicly reported pretraining experiments range from 1B to 600B parameters; GPU-friendly formulation (BDH-GPU) uses ReLU-low-rank transformations with linear attention, deployed via Amazon SageMaker HyperPod.

LIMITATIONS AND OPEN QUESTIONS

Interference accumulates: when many tokens share similar keys, writes collide in the shared matrix and retrieval degrades. BDH's sparse activation reduces but does not eliminate this. State capacity is fundamentally bounded by d^2 — sufficiently dense or long sequences will saturate it. Cross-session durable learning (consolidating fast state into slow weights) remains unsolved. BDH's reported results are developer-reported; no independent reproduction of the full system is publicly available as of this submission.

PRIMARY SOURCES

- [1] Katharopoulos et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML 2020. arXiv:2006.16236.
- [2] Pathway (2024). Dragon Hatchling (BDH): A Brain-Inspired Post-Transformer Architecture. Pathway Technical Report.
- [3] Pathway (2024). BDH-CQ: In-Context Learning via Recurrent State. Pathway Technical Report.
- [4] Dao & Gu (2024). Transformers are SSMS: Generalized Models with Structured State Spaces. NeurIPS 2024. arXiv:2405.21060.
- [5] Schlag et al. (2021). Linear Transformers are Secretly Fast Weight Programmers. ICML 2021. arXiv:2102.11174.
- [6] Sun et al. (2023). Retentive Network: A Successor to Transformer for LLMs. arXiv:2307.08621.